The Pangea Puzzle, coded in Julia (Tutorial)

Jeet Sukumaran and Maritza Sardina

Table of contents

The original "Pangea Puzzle" tutorial was made by Mark D. Uhen from George Mason University, and was originally published on April 17th, 2017. This tutorial incorporates the original tutorial using a mix of both Julia code and written questions, as well as use of the PaleobiologyDB's Navigator tool as a reference. The original tutorial can be accessed at https://serc.carleton.edu/NAGTWorkshops/intro/activities/177872.html.

How can we better incorporate data on the distributions of certain organisms on the modern-day continents? (Uhen, 2017)

Recent studies involving paleontological datasets and records have started to incorporate high-quality, accessible, and expansive databases involving the fossil occurrences, taxonomic ranks, and geospatial data of all sorts of organisms. Specifically, with the emergence of resources like PaleobiologyDB, its Navigator tool, and other Internet resources, it is now easier to map out the geographical and stratigraphical appearances of major groups of organisms on both past-day and modern-day continents with fossil records/datasets. Specifically, we will be analyzing the occurrences of certain fossils belonging to several groups of organisms.

The main goal of this tutorial is to develop a better understanding of how fossil occurrences can be mapped out on the modern-day continents and to gain experience working with Julia. In particular, Julia will be used to create datasets (in the form of DataFrames) and high-quality visualizations of our data, specifically to better interpret how certain fossils appear on today's map.

1 The Basics of Julia and PaleobiologyDB

1.1 Julia

Julia is a dynamic, high-level, and open-source programming language that incorporates multiple dispatch, just-in-time compilation, and a rich ecosystem of packages. Julia is also designed for high-performance numerical and scientific computing, which makes it an excellent choice for data analysis, machine learning, and scientific research. Julia is widely supported by a community of programmers and programmer tools, including Jupyter notebooks, Visual Studio Code (VSCode), and more. Julia's syntax is designed to be accessible to both beginning and experienced programmers, as it is easy to read and write.

Before starting the tutorial, it is crucial to have all the necessary packages installed. In your terminal, you will use Julia's built-in package manager "Pkg" to install several packages, using the command "Pkg.add", with the names of each package inside the brackets and in quotation marks. For the purpose of this tutorial, you will need to install: "CairoMakie", "ColorSchemes," "DataFrames," "GeoMakie," "Makie," and "PaleobiologyDB." We'll go a little deeper into each package throughout the tutorial.

```julia
using Pkg
Pkg.add(["PaleobiologyDB", "DataFrames", "Makie", "CairoMakie", "GeoMakie", "ColorSchemes"])
```

After installing these packages, you can use them using the command "using" in your Julia REPL, with the package names inserted after the command, as shown below:

```julia
using PaleobiologyDB, PaleobiologyDB.Taxonomy
using DataFrames, Makie, CairoMakie, GeoMakie, ColorSchemes
```

2 Deep dive into Julia packages

2.1 DataFrames

"DataFrames" is a Julia package that allows you to acquire and work with tabular data, and it functions as a general-purpose data science and statistics tool. As a package, it has numerous programming language equivalents, such as Python's "pandas" package and R's array of data packages, including "data.table," "dplyr," and "data.frame". "DataFrames" is essential in the Julia Data ecosystem, as it serves an essential purpose in data manipulation, cleaning, and statistical analysis. For the purpose of this tutorial, this package can be used to create and generate tables of fossil occurrence data and other types of data stemming from PaleobiologyDB.

2.2 Makie family and ColorSchemes

"Makie" and "CairoMakie" are visualization packages that allow programmers to create high-quality plots, maps, and visualizations of data in Julia. "GeoMakie" is another example of a "Makie-esque" package that is designed for plotting geospatial data through maps and geographical distribution plots. "ColorSchemes" is a color-based package that provides a variety of color palettes and schemes that can be used to enhance the appearance, aesthetics, and clarity of high-quality visualizations that are created using the "Makie" family packages.

NOTE: For Makie-made plots and visualizations, you can check out https://beautiful.makie.org/dev/ for more information on how to customize your visualizations, including how to use different kinds of color schemes, plot attributes (e.g., labels, ticks, fonts, etc.), and more. Beautiful Makie is a useful resource for learning how to construct high-quality, "beautiful" visualizations in Julia using the "Makie" family of packages, including "Makie," "CairoMakie," "GeoMakie," and much more.

2.3 PaleobiologyDB

PaleobiologyDB is a public database that contains information on fossil occurrences, including their geographic locations, ages, and taxonomic classifications. This database seeks to provide a comprehensive and accessible resource for paleontologists, researchers, educators, and the general public interested in the field of paleontology. Additionally, it contains a complete catalogue of all fossil occurrences through geological time and the tree of life.

For this tutorial, we will be using the “PaleobiologyDB.jl” package, which is a Julia interface to the PaleobiologyDB database. This package allows you to access the database and acquire specific fossil occurrences based on various factors, including taxonomic groups (e.g., genera, families, classes, etc.), geographic location, appearance data, and time intervals.

“PaleobiologyDB.jl” can be downloaded from GitHub via the “Pkg” package manager in Julia, via the website: https://github.com/jeetsukumaran/PaleobiologyDB.jl or by using the following code in your Julia REPL:

```
using Pkg
Pkg.add("PaleobiologyDB")
```

If you go on PaleobiologyDB’s official website at https://paleobiodb.org/#/, you can click on the “Learn” tab (in red), then click on “Teach and Learn”. On the Resources page, you can scroll through both web apps and tutorials using the PaleobiologyDB database. For the sake of this tutorial, we will be using the “Pangea Puzzle” tutorial, which is under “Lesson Plans and Educational Activities”. Clicking on “The Pangea Puzzle” takes you to the official activity PDF, which you can download from https://serc.carleton.edu/NAGTWorkshops/intro/activities/177872.html. This activity PDF contains instructions and questions that will be answered throughout the tutorial. You can use both the PDF and the Navigator (at https://paleobiodb.org/navigator/) alongside your Julia tutorial.

3 Part 1: Construct a map of fossil distributions on the modern continents using Lystrosaurus.

3.1 Creating a general map of Lystrosaurus fossil occurrences

After setting up your packages, you will need to query PaleobiologyDB’s database using ‘pbdb_occurrences’ to retrieve the fossil occurrences of *Lystrosaurus*. This is to acquire the data necessary for plotting the distribution of *Lystrosaurus* fossils on a map.

```
animal = "Lystrosaurus"

lystro_raw = pbdb_occurrences(
    base_name = animal,
    show = ["coords", "classext"],
    vocab = "pbdb",
    extids = true
)
```

Using 'println' can help you print out and understand the data that you acquired from the query, including the number of fossil occurrences of *Lystrosaurus*.

```
println("Found ", nrow(df_raw), " Lystrosaurus fossils!")
```

The output will read: "Found 124 Lystrosaurus fossils!"

This indicates that you have found exactly 124 fossil occurrences of *Lystrosaurus* in the database, which can be used to create a map of fossil distributions of *Lystrosaurus*.

We'll clean up the data next. This can be done by doing the following:

```
clean = dropmissing(df_raw, [:lng, :lat, :min_ma, :max_ma])

clean = transform(
    clean,
    [:min_ma, :max_ma] => ((lo, hi) -> (lo .+ hi) ./ 2) => :mid_ma
)
```

Ensure that you can print out the cleaned data, specifically to clarify that the data has been properly "cleaned."

```
println("Records with valid coordinates: ", nrow(clean))
println("Records dropped: ", nrow(df_raw) - nrow(clean))
```

The output will now read: "Records with valid coordinates: 124" and "Records dropped: 0" respectively. These aspects represent the fossil records that contain valid coordinates and the exact number of fossil records that had to be dropped due to missing data, which was 0.

After cleaning the data some more, you can use the data to make a map of Lystrosaurus fossil occurrences using the "CairoMakie" and "GeoMakie" packages, which allow you to create high-quality visualizations and maps, particularly of geological data using the PaleobiologyDB database in Julia. You can use 'Figure' to create a new figure. 'GeoAxis' can be used to set up a geographic axis with specific projections. 'poly!' can be used to add land polygons to the map, while 'scatter!' can be used to plot the fossil occurrences as points on the map, colored by their age. Finally, you can use 'Colorbar' to indicate the age of the fossils.

NOTE: https://docs.juliaplots.org/dev/ is a useful resource for learning how to plot data in Julia, using packages such as "Plots," "CairoMakie," "ColorSchemes" (for colors), and more. You can use the table of contents to find specific topics related to plotting in Julia, and you can use plot attributes to customize/clean up your visualizations. If you're using Visual Studio Code (VSCode), you can use Copilot to look up specific attributes for functions in plotting.

The code for making the map is as follows:

```
fig = Figure(size = (1000, 600))
ga = GeoAxis(
    fig[1,1],
    dest = "proj=natearth2",
    title = "Lystrosaurus fossil occurrences"
)

poly!(ga, GeoMakie.land(); color = :whitesmoke,
    strokecolor = :gray55, strokewidth = 0.5)

sc = scatter!(ga, clean.lng, clean.lat; color = clean.mid_ma,
    colormap = :thermal, markersize = 7.5)

Colorbar(fig[1, 2], sc; label = "Age (Ma)", flipaxis = false)

display(fig)

save("lystrosaurus_pangea_puzzle_occurrences_072326.png", fig)
```

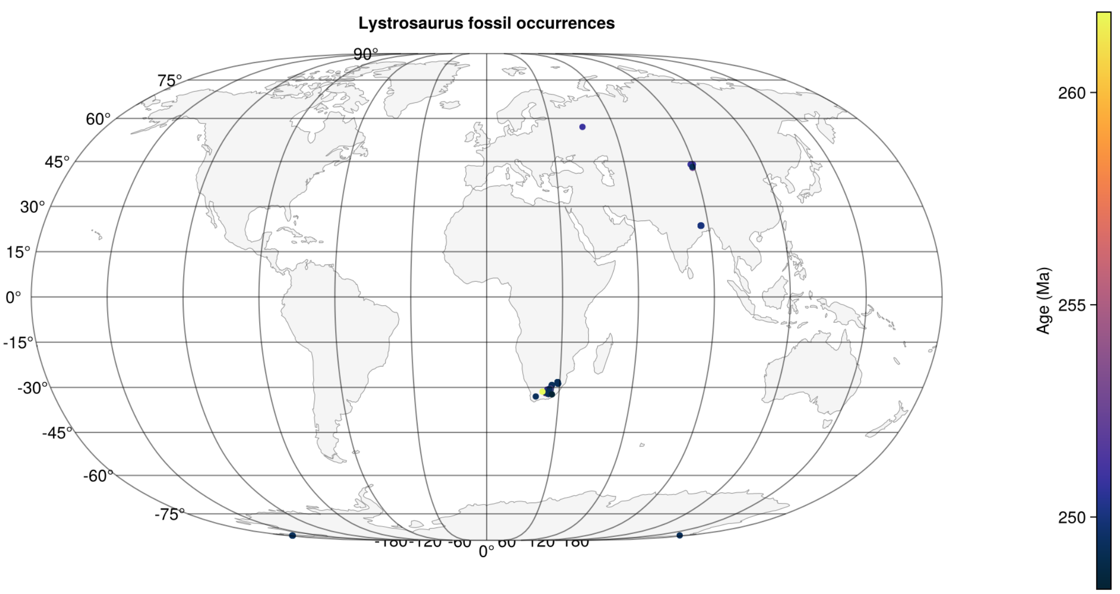

Figure 1: Lystrosaurus fossil occurrences

This code will create a map of *Lystrosaurus* fossil occurrences, with the points colored by their age (in millions of years). In this case, *Lystrosaurus* fossils were mainly found in South Africa, but also through Asia and Antarctica. For reference, you'll also need to answer the questions that come with the original tutorial.

Question 1: In one to three sentences, describe how the occurrences of *Lystrosaurus* fossils are distributed on the modern map. if you were to redraw this map during the time that *Lystrosaurus* lived, what do you predict will happen to the distribution of *Lystrosaurus* fossil localites? [Uhen, 2017]

Question 2: Based on the matching on your modern map, when did *Lystrosaurus* live? [Uhen, 2017]

3.2 Creating a specific map of Lystrosaurus fossil occurrences in a specific time interval (When did they live?)

Based on your answer to question 2, you can reconstruct your map to show the distribution of *Lystrosaurus* fossils during the time that they lived. Using "pbdb_occurrences" again, you can acquire data on *Lystrosaurus* fossil occurrences during the time that they lived by adding "interval" to the function. In this example, we'll reuse the workflow previously established using the "Triassic" interval.

```
animal = "Lystrosaurus"
period = "Early Triassic"

lystro_triassic_raw = pbdb_occurrences(
    base_name = animal,
    interval = period,
    show = ["coords", "classext"],
    vocab = "pbdb",
    extids = true
)

println("Found ", nrow(lystro_triassic_raw), " Early Triassic Lystrosaurus fossils!")

clean = dropmissing(lystro_triassic_raw, [:lng, :lat, :min_ma, :max_ma])

clean = transform(
    clean,
    [:min_ma, :max_ma] => ((lo, hi) -> (lo .+ hi) ./ 2) => :mid_ma
)

println("Records with valid coordinates: ", nrow(clean))
println("Records dropped: ", nrow(lystro_triassic_raw) - nrow(clean))
```

```
fig = Figure(size = (1000, 600))
ga = GeoAxis(
    fig[1,1],
    dest = "proj=natearth2",
    title = "Triassic Lystrosaurus fossil occurrences"
)

poly!(ga, GeoMakie.land(); color = :whitesmoke,
    strokecolor = :gray55, strokewidth = 0.5)

sc = scatter!(ga, clean.lng, clean.lat; color = clean.mid_ma,
    colormap = :acton, markersize = 7.5)

Colorbar(fig[1, 2], sc; label = "Age (Ma)", flipaxis = false)

display(fig)

save("early_triassic_lystrosaurus_pangea_puzzle_occurrences_072326.png", fig)
```

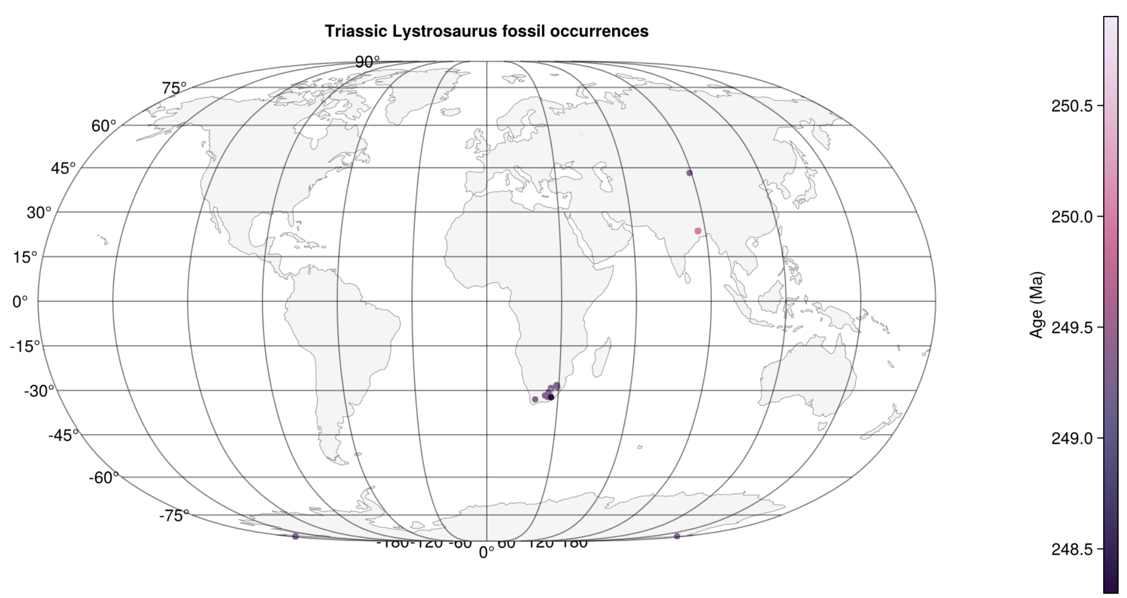

Figure 2: Triassic Lystrosaurus fossil occurrences

Question 3: In one to three sentences, describe any changes in the distribution of your fossil. Make sure to save a copy of your map for reference. [Uhen, 2017]

4 Part 2: Construct a map of fossil distributions of Mesosaurus and Glossopteris on the modern continents.

4.1 Creating a general map of Mesosaurus fossil occurrences

Using the workflow established in Part 1, you will construct maps of *Mesosaurus* fossil occurrences on the modern continents. After creating the map, take a closer look at the distribution of *Mesosaurus* fossils and answer the following questions in your activity PDF.

Example workflow for *Mesosaurus*:

```
animal = "Mesosaurus"

mesosaurus_raw = pbdb_occurrences(
    base_name = animal,
    show = ["coords", "classext"],
    vocab = "pbdb",
    extids = true
)

println("Found ", nrow(mesosaurus_raw), " Mesosaurus fossils!")

clean = dropmissing(mesosaurus_raw, [:lng, :lat, :min_ma, :max_ma])

clean = transform(
    clean,
    [:min_ma, :max_ma] => ((lo, hi) -> (lo .+ hi) ./ 2) => :mid_ma
)

println("Records with valid coordinates: ", nrow(clean))
println("Records dropped: ", nrow(mesosaurus_raw) - nrow(clean))

fig = Figure(size = (1000, 600))
ga = GeoAxis(
    fig[1,1],
    dest = "proj=natearth2",
    title = "Mesosaurus fossil occurrences"
)
```

```
poly!(ga, GeoMakie.land(); color = :whitesmoke,
    strokecolor = :gray60, strokewidth = 0.75)

sc = scatter!(ga, clean.lng, clean.lat; color = clean.mid_ma,
    colormap = :lapaz, markersize = 6.5)

Colorbar(fig[1, 2], sc; label = "Age (Ma)", flipaxis = false)

display(fig)

save("mesosaurus_pangea_puzzle_occurrences_072326.png", fig)
```

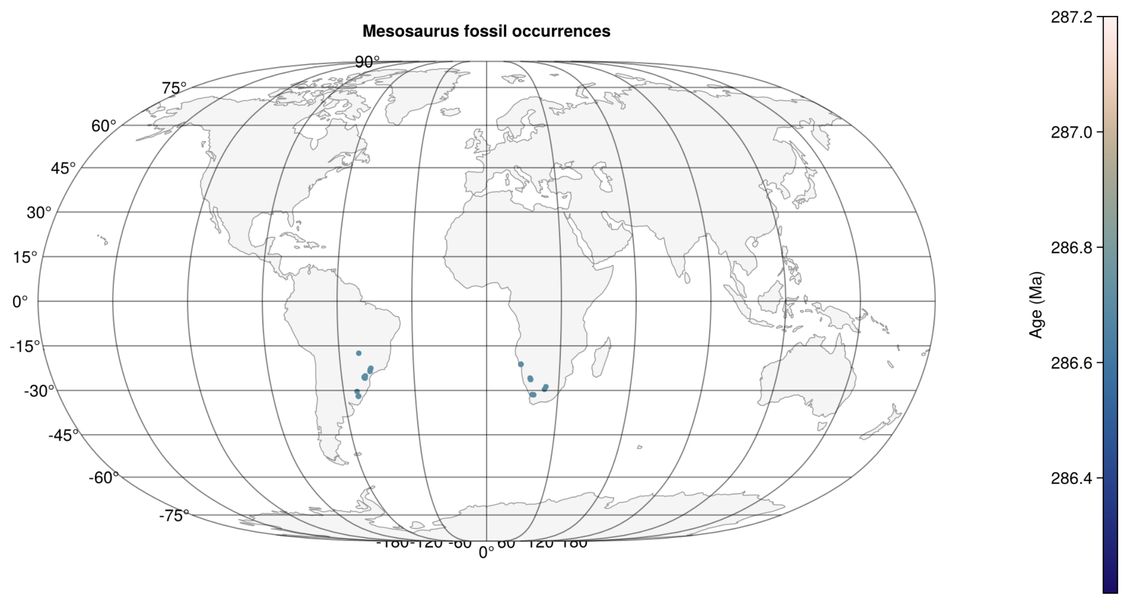

Figure 3: Mesosaurus fossil occurrences

Question 1: Look at the distribution for these fossils. Describe the distribution of this fossil on the land masses. [Uhen, 2017]

Question 2: How do you think this animal might have gotten distributed in this pattern? [Uhen, 2017]

Question 3: Using online resources and your activity PDF, look up some information on *Mesosaurus*. Now that you know what kind of animal it is, does your answer to number 3 still make sense? Additionally, you can use “pbdb_occurrences” to acquire more data on *Mesosaurus* using a description workflow. For example:

```
mesosaurus_occs = pbdb_occurrences(
    ;
    base_name = "Mesosaurus",
    show = "full",
    vocab = "pbdb",
    extids = true,
    limit = 1000
)

typeof(mesosaurus_occs)

size(mesosaurus_occs)

foreach(println, names(mesosaurus_occs))

describe(mesosaurus_occs)

show(describe(mesosaurus_occs), allrows = true)

combine(groupby(mesosaurus_occs, :accepted_rank), nrow)
```

Question 4a: When did *Mesosaurus* live? [Uhen, 2017]

We can calculate the range of *Mesosaurus* fossils using “pbdb_occurrences,” using the following code:

```
using PaleobiologyDB, PaleobiologyDB.Taxonomy
using DataFrames, Makie, CairoMakie, GeoMakie, ColorSchemes

PaleobiologyDB.set_autocaching!(true);

animal = "Mesosaurus"

df_meso_raw = pbdb_occurrences(
    base_name = animal,
    show = "full",
    vocab = "pbdb",
    extids = true
)
```

```
df_meso = dropmissing(df_meso_raw, [
    :genus,
    :early_interval,
    :max_ma,
    :min_ma
])
```

To drop unresolved or unqualified taxa, you can use the following code:

```
df_meso = drop_unresolved_taxa(df_meso, :genus)
```

To calculate a midpoint age (Ma) for each fossil occurrence, you can then use the following code:

```
df_meso.midpoint_age = (df_meso.max_ma .+ df_meso.min_ma) ./ 2
```

The split-apply-combine method is a type of workflow that splits the bigger DataFrame (consisting of all fossil occurrences) into per-genus specific tables consisting of first appearance and last appearance data. This is done by applying a summary function to each sub-table and combining them to form a new DataFrame, as follows:

```
appearance_df_meso = combine(
    groupby(df_meso, :genus, sort = true),
    :midpoint_age => maximum => :first_appearance_datum,
    :midpoint_age => minimum => :last_appearance_datum
)
appearance_df_meso = sort(appearance_df_meso, :first_appearance_datum, rev = true)
```

To create a chart of this range, you can use the "Makie" and "CairoMakie" packages to create a map and bar chart of the first and last appearance data for each genus, using the following code:

```
using Makie, CairoMakie
fig = Figure()
ax = Axis(
    fig[1,1],
    xlabel = "Midpoint age (Ma)",
    ylabel = "Genus",
    yticks = (1:nrow(appearance_df_meso), appearance_df_meso.genus),
    xreversed = true
```

```
)

for i in 1:nrow(appearance_df_meso)
    x1 = appearance_df_meso.first_appearance_datum[i]
    x2 = appearance_df_meso.last_appearance_datum[i]
    lines!(ax, [x1, x2], [i, i])
end

fig

save("mesosaurus_range_through_072326.png", fig)
```

Question 4b: Looking at the map you created, what is different about your map? Does it make more sense now in light of what you know about Mesosaurus? Why or why not? [Uhen, 2017]

NOTE: If nothing appears on your graph, you can check back to your sub-table that was made using the split-apply-combine method to make sure that the data was properly grouped and summarized. You can also check to make sure that the "first_appearance_datum" and "last_appearance_datum" columns were properly created with your data. Both of these aspects can be analyzed to determine if we have too little data or if there are any errors in your workflow. Additionally, we may consider the differences between coordinate spacing and GIS spacing, when graphing your data. This may affect how certain data is plotted on your chart.

4.2 Creating a general map of Glossopteris fossil occurrences

Repeat the procedure/workflow used for *Mesosaurus* above, this time for the genus *Glossopteris*. After constructing the map for *Glossopteris*, make sure to save a copy of your map for reference, especially in answering the question below.

Question 5: What do all of these fossil distribution suggest? [Uhen, 2017]

5 Part 3: Construct a map of fossil distributions of Marsupialia on the modern continents and previous time periods.

Here is an example of a complete workflow (of constructing occurrence maps) for the infraclass Marsupialia in the Neogene time period, which you can use as a template to create your own map of fossil distributions for Marsupialia on the modern continents and other time periods. You can also use this workflow to create maps for other organisms, as shown in Part 4 of the tutorial.

```
animal = "Marsupialia"
period = "Neogene"

marsupialia_neogene_raw = pbdb_occurrences(
    base_name = animal,
    interval = period,
    show = ["coords", "classext"],
    vocab = "pbdb",
    extids = true
)

println("Found ", nrow(marsupialia_neogene_raw), " Neogene Marsupialia fossils!")

clean_marsu = dropmissing(marsupialia_neogene_raw, [:lng, :lat, :min_ma, :max_ma])
clean_marsu = transform(
    clean_marsu,
    [:min_ma, :max_ma] => ((lo, hi) -> (lo .+ hi) ./ 2) => :mid_ma
)

println("Records with valid coordinates: ", nrow(clean_marsu))
println("Records dropped: ", nrow(marsupialia_neogene_raw) - nrow(clean_marsu))

fig = Figure(size = (1000, 600))
ga = GeoAxis(
    fig[1,1],
    dest = "proj=natearth2",
    title = "Marsupialia fossil occurrences in the Neogene"
)

poly!(ga, GeoMakie.land(); color = :whitesmoke,
    strokecolor = :gray55, strokewidth = 0.5)

sc = scatter!(ga, clean_marsu.lng, clean_marsu.lat; color = clean_marsu.mid_ma,
    colormap = :thermal, markersize = 7.5)

Colorbar(fig[1, 2], sc; label = "Age (Ma)", flipaxis = false)

display(fig)

save("marsupialia_neogene_occurrences_072326.png", fig)
```

After completing the map for Marsupialia fossil occurrences in the Neogene, work back in

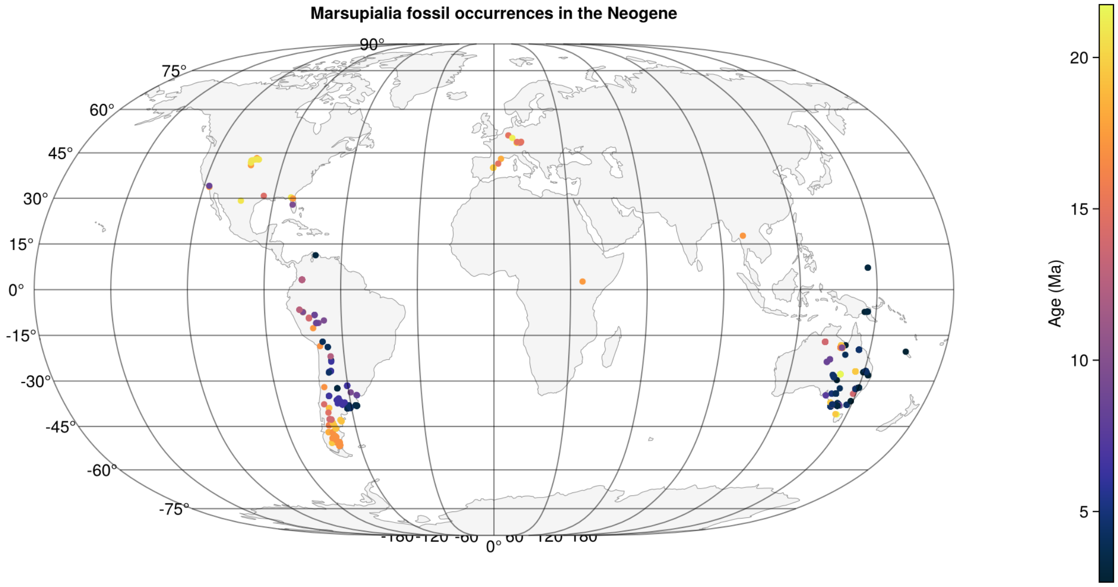

Figure 4: Neogene Marsupialia fossil occurrences

time and construct maps for the following time periods: Paleogene, Cretaceous, Jurassic, and Triassic. After creating the maps, answer the following questions in your activity PDF:

Question 1: What is the most recent time period when the continents, including Marsupialia, were connected? [Uhen, 2017]

Question 2: How does this help explain the current distribution of Marsupialia, specifically in Australia, South America, and North America? [Uhen, 2017]

6 Part 4: Choose your own fossil!

Using the same workflow indicated in Part 3, choose a fossil of your choice and construct a map of its distribution. If you are having trouble thinking of a fossil, you can use fossils such as those from dinosaurs, early mammals, or organisms in marine life (i.e., whales, dolphins, etc.). If you don't get many dots or "hits" on your map, try expanding your search to include a bigger group of organisms, such as families, orders, or classes.

As such, answer the following questions below, which are also included in the original tutorial PDF:

Question 1: Do you have the same predictions for how the distributions will change from the modern map to the paleo-time map for both the marine and terrestrial organisms? Why might the marine and terrestrial organism distributions behave differently? Make sure to save a copy of your map for reference. [Uhen, 2017]

Question 2: What did you learn about the past distributions of your fossils? Are these distributions what you expected, or were you surprised in any way? If so, what surprised you? [Uhen, 2017]

7 References (for Original Activity)

Uhen, M. (2017, April 17). The Pangea Puzzle. Cutting Edge. https://serc.carleton.edu/NAGT-Workshops/intro/activities/177872.html